

Auto MPG Dataset: Column Explanations

mpg Miles per gallon (MPG), a measure of fuel efficiency for the car. Higher values indicate better fuel economy.

cylinders The number of cylinders in the engine, typically 4, 6, or 8. More cylinders generally indicate a more powerful engine.

displacement Engine displacement in cubic inches. It refers to the total volume of air/fuel mixture an engine can draw in during one cycle.

horsepower The engine's power output, measured in horsepower. Higher values indicate more powerful engines.

weight The vehicle's weight in pounds. Heavier vehicles generally have lower fuel efficiency.

acceleration The time it takes for the car to accelerate from 0 to 60 mph, measured in seconds. Lower values indicate quicker acceleration.

model year The year the car model was released. Ranges from 1970 to 1982 in the dataset.

origin The region where the car was manufactured. 1 = USA, 2 = Europe, 3 = Japan.

car name The name of the car model.

```
In [2]: 1 import pandas as pd
        2 import numpy as np
        3 import matplotlib.pyplot as plt
        4 import seaborn as sns
```

```
In [3]: 1 df=pd.read_csv(r'D:\Data Science_Data Analyst\Arbor_Class\Assigenments\au
        2 df.head()
```

```
Out[3]:
```

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	car name
0	18.0	8	307.0	130	3504	12.0	70	1	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	1	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	1	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	1	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	1	ford torino

```
In [4]: 1 df.shape
```

```
Out[4]: (398, 9)
```

Observations

This data set has 398 rows and 9 columns

In [5]: 1 df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 398 entries, 0 to 397
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   mpg              398 non-null    float64
1   cylinders         398 non-null    int64
2   displacement      398 non-null    float64
3   horsepower        398 non-null    object
4   weight            398 non-null    int64
5   acceleration      398 non-null    float64
6   model year       398 non-null    int64
7   origin            398 non-null    int64
8   car name         398 non-null    object
dtypes: float64(3), int64(4), object(2)
memory usage: 28.1+ KB
```

Observations

This dataset has 9 columns out of which 4 are int, 3 are float and 2 are object data types

Data quality issues

1. type casting
2. Noisy or inconsistent
3. Missing values
4. Duplicates
5. Outliers

. Noisy or inconsisten

In [6]: 1 df.columns

```
Out[6]: Index(['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
              'acceleration', 'model year', 'origin', 'car name'],
              dtype='object')
```

```
In [7]: 1 df['mpg'].unique()
```

```
Out[7]: array([18. , 15. , 16. , 17. , 14. , 24. , 22. , 21. , 27. , 26. , 25. ,
          10. , 11. ,  9. , 28. , 19. , 12. , 13. , 23. , 30. , 31. , 35. ,
          20. , 29. , 32. , 33. , 17.5, 15.5, 14.5, 22.5, 24.5, 18.5, 29.5,
          26.5, 16.5, 31.5, 36. , 25.5, 33.5, 20.5, 30.5, 21.5, 43.1, 36.1,
          32.8, 39.4, 19.9, 19.4, 20.2, 19.2, 25.1, 20.6, 20.8, 18.6, 18.1,
          17.7, 27.5, 27.2, 30.9, 21.1, 23.2, 23.8, 23.9, 20.3, 21.6, 16.2,
          19.8, 22.3, 17.6, 18.2, 16.9, 31.9, 34.1, 35.7, 27.4, 25.4, 34.2,
          34.5, 31.8, 37.3, 28.4, 28.8, 26.8, 41.5, 38.1, 32.1, 37.2, 26.4,
          24.3, 19.1, 34.3, 29.8, 31.3, 37. , 32.2, 46.6, 27.9, 40.8, 44.3,
          43.4, 36.4, 44.6, 40.9, 33.8, 32.7, 23.7, 23.6, 32.4, 26.6, 25.8,
          23.5, 39.1, 39. , 35.1, 32.3, 37.7, 34.7, 34.4, 29.9, 33.7, 32.9,
          31.6, 28.1, 30.7, 24.2, 22.4, 34. , 38. , 44. ])
```

```
In [8]: 1 df['mpg'].nunique()
```

```
Out[8]: 129
```

```
In [9]: 1 df['mpg'].unique()
```

```
Out[9]: array([18. , 15. , 16. , 17. , 14. , 24. , 22. , 21. , 27. , 26. , 25. ,
          10. , 11. ,  9. , 28. , 19. , 12. , 13. , 23. , 30. , 31. , 35. ,
          20. , 29. , 32. , 33. , 17.5, 15.5, 14.5, 22.5, 24.5, 18.5, 29.5,
          26.5, 16.5, 31.5, 36. , 25.5, 33.5, 20.5, 30.5, 21.5, 43.1, 36.1,
          32.8, 39.4, 19.9, 19.4, 20.2, 19.2, 25.1, 20.6, 20.8, 18.6, 18.1,
          17.7, 27.5, 27.2, 30.9, 21.1, 23.2, 23.8, 23.9, 20.3, 21.6, 16.2,
          19.8, 22.3, 17.6, 18.2, 16.9, 31.9, 34.1, 35.7, 27.4, 25.4, 34.2,
          34.5, 31.8, 37.3, 28.4, 28.8, 26.8, 41.5, 38.1, 32.1, 37.2, 26.4,
          24.3, 19.1, 34.3, 29.8, 31.3, 37. , 32.2, 46.6, 27.9, 40.8, 44.3,
          43.4, 36.4, 44.6, 40.9, 33.8, 32.7, 23.7, 23.6, 32.4, 26.6, 25.8,
          23.5, 39.1, 39. , 35.1, 32.3, 37.7, 34.7, 34.4, 29.9, 33.7, 32.9,
          31.6, 28.1, 30.7, 24.2, 22.4, 34. , 38. , 44. ])
```

```
In [10]: 1 df['cylinders'].unique()
```

```
Out[10]: array([8, 4, 6, 3, 5], dtype=int64)
```

```
In [11]: 1 df['displacement'].unique()
```

```
Out[11]: array([307. , 350. , 318. , 304. , 302. , 429. , 454. , 440. , 455. ,
          390. , 383. , 340. , 400. , 113. , 198. , 199. , 200. ,  97. ,
          110. , 107. , 104. , 121. , 360. , 140. ,  98. , 232. , 225. ,
          250. , 351. , 258. , 122. , 116. ,  79. ,  88. ,  71. ,  72. ,
           91. ,  97.5,  70. , 120. ,  96. , 108. , 155. ,  68. , 114. ,
          156. ,  76. ,  83. ,  90. , 231. , 262. , 134. , 119. , 171. ,
          115. , 101. , 305. ,  85. , 130. , 168. , 111. , 260. , 151. ,
          146. ,  80. ,  78. , 105. , 131. , 163. ,  89. , 267. ,  86. ,
          183. , 141. , 173. , 135. ,  81. , 100. , 145. , 112. , 181. ,
          144. ])
```

```
In [12]: 1 df['horsepower'].unique()
```

```
Out[12]: array(['130', '165', '150', '140', '198', '220', '215', '225', '190',
                '170', '160', '95', '97', '85', '88', '46', '87', '90', '113',
                '200', '210', '193', '?', '100', '105', '175', '153', '180', '110',
                '72', '86', '70', '76', '65', '69', '60', '80', '54', '208', '155',
                '112', '92', '145', '137', '158', '167', '94', '107', '230', '49',
                '75', '91', '122', '67', '83', '78', '52', '61', '93', '148',
                '129', '96', '71', '98', '115', '53', '81', '79', '120', '152',
                '102', '108', '68', '58', '149', '89', '63', '48', '66', '139',
                '103', '125', '133', '138', '135', '142', '77', '62', '132', '84',
                '64', '74', '116', '82'], dtype=object)
```

```
In [33]: 1 df['weight'].unique()
```

```
Out[33]: array([3504, 3693, 3436, 3433, 3449, 4341, 4354, 4312, 4425, 3850, 3563,
                3609, 3761, 3086, 2372, 2833, 2774, 2587, 2130, 1835, 2672, 2430,
                2375, 2234, 2648, 4615, 4376, 4382, 4732, 2264, 2228, 2046, 2634,
                3439, 3329, 3302, 3288, 4209, 4464, 4154, 4096, 4955, 4746, 5140,
                2962, 2408, 3282, 3139, 2220, 2123, 2074, 2065, 1773, 1613, 1834,
                1955, 2278, 2126, 2254, 2226, 4274, 4385, 4135, 4129, 3672, 4633,
                4502, 4456, 4422, 2330, 3892, 4098, 4294, 4077, 2933, 2511, 2979,
                2189, 2395, 2288, 2506, 2164, 2100, 4100, 3988, 4042, 3777, 4952,
                4363, 4237, 4735, 4951, 3821, 3121, 3278, 2945, 3021, 2904, 1950,
                4997, 4906, 4654, 4499, 2789, 2279, 2401, 2379, 2124, 2310, 2472,
                2265, 4082, 4278, 1867, 2158, 2582, 2868, 3399, 2660, 2807, 3664,
                3102, 2875, 2901, 3336, 2451, 1836, 2542, 3781, 3632, 3613, 4141,
                4699, 4457, 4638, 4257, 2219, 1963, 2300, 1649, 2003, 2125, 2108,
                2246, 2489, 2391, 2000, 3264, 3459, 3432, 3158, 4668, 4440, 4498,
                4657, 3907, 3897, 3730, 3785, 3039, 3221, 3169, 2171, 2639, 2914,
                2592, 2702, 2223, 2545, 2984, 1937, 3211, 2694, 2957, 2671, 1795,
                2464, 2572, 2255, 2202, 4215, 4190, 3962, 3233, 3353, 3012, 3085,
                2035, 3651, 3574, 3645, 3193, 1825, 1990, 2155, 2565, 3150, 3940,
                3270, 2930, 3820, 4380, 4055, 3870, 3755, 2045, 1945, 3880, 4060,
                4140, 4295, 3520, 3425, 3630, 3525, 4220, 4165, 4325, 4335, 1940,
                2740, 2755, 2051, 2075, 1985, 2190, 2815, 2600, 2720, 1800, 2070,
                3365, 3735, 3570, 3535, 3155, 2965, 3430, 3210, 3380, 3070, 3620,
                3410, 3445, 3205, 4080, 2560, 2230, 2515, 2745, 2855, 2405, 2830,
                3140, 2795, 2135, 3245, 2990, 2890, 3265, 3360, 3840, 3725, 3955,
                3830, 4360, 4054, 3605, 1925, 1975, 1915, 2670, 3530, 3900, 3190,
                3420, 2200, 2150, 2020, 2595, 2700, 2556, 2144, 1968, 2120, 2019,
                2678, 2870, 3003, 3381, 2188, 2711, 2434, 2110, 2800, 2085, 2335,
                2950, 3250, 1850, 2145, 1845, 2910, 2420, 2500, 2905, 2290, 2490,
                2635, 2620, 2725, 2385, 1755, 1875, 1760, 2050, 2215, 2380, 2320,
                2210, 2350, 2615, 3230, 3160, 2900, 3415, 3060, 3465, 2605, 2640,
                2575, 2525, 2735, 2865, 3035, 1980, 2025, 1970, 2160, 2205, 2245,
                1965, 1995, 3015, 2585, 2835, 2665, 2370, 2790, 2295, 2625],
                dtype=int64)
```

In [13]: 1 df['acceleration'].unique()

Out[13]: array([12. , 11.5, 11. , 10.5, 10. , 9. , 8.5, 8. , 9.5, 15. , 15.5, 16. , 14.5, 20.5, 17.5, 12.5, 14. , 13.5, 18.5, 19. , 13. , 19.5, 18. , 17. , 23.5, 16.5, 21. , 16.9, 14.9, 17.7, 15.3, 13.9, 12.8, 15.4, 17.6, 22.2, 22.1, 14.2, 17.4, 16.2, 17.8, 12.2, 16.4, 13.6, 15.7, 13.2, 21.9, 16.7, 12.1, 14.8, 18.6, 16.8, 13.7, 11.1, 11.4, 18.2, 15.8, 15.9, 14.1, 21.5, 14.4, 19.4, 19.2, 17.2, 18.7, 15.1, 13.4, 11.2, 14.7, 16.6, 17.3, 15.2, 14.3, 20.1, 24.8, 11.3, 12.9, 18.8, 18.1, 17.9, 21.7, 23.7, 19.9, 21.8, 13.8, 12.6, 16.1, 20.7, 18.3, 20.4, 19.6, 17.1, 15.6, 24.6, 11.6])

In [14]: 1 df['model year'].unique()

Out[14]: array([70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82], dtype=int64)

In [15]: 1 df['origin'].unique()

Out[15]: array([1, 3, 2], dtype=int64)

In [16]: 1 df['car name'].unique()

"plymouth 'cuda 340", 'chevrolet monte carlo',
 'buick estate wagon (sw)', 'toyota corona mark ii',
 'plymouth duster', 'amc hornet', 'ford maverick', 'datsun pl510',
 'volkswagen 1131 deluxe sedan', 'peugeot 504', 'audi 100 ls',
 'saab 99e', 'bmw 2002', 'amc gremlin', 'ford f250', 'chevy c20',
 'dodge d200', 'hi 1200d', 'chevrolet vega 2300', 'toyota corona',
 'ford pinto', 'plymouth satellite custom', 'ford torino 500',
 'amc matador', 'pontiac catalina brougham', 'dodge monaco (sw)',
 'ford country squire (sw)', 'pontiac safari (sw)',
 'amc hornet sportabout (sw)', 'chevrolet vega (sw)',
 'pontiac firebird', 'ford mustang', 'mercury capri 2000',
 'opel 1900', 'peugeot 304', 'fiat 124b', 'toyota corolla 1200',
 'datsun 1200', 'volkswagen model 111', 'plymouth cricket',
 'toyota corona hardtop', 'dodge colt hardtop', 'volkswagen type
 3',
 'chevrolet vega', 'ford pinto runabout', 'amc ambassador sst',
 'mercury marquis', 'buick lesabre custom',
 'oldsmobile delta 88 royale', 'chrysler newport royal',
 'mazda rx2 coupe', 'amc matador (sw)',
 'chevrolet chevelle concours (sw)', 'ford gran torino (sw)',

In [17]: 1 df['horsepower'].unique()

Out[17]: array(['130', '165', '150', '140', '198', '220', '215', '225', '190',
 '170', '160', '95', '97', '85', '88', '46', '87', '90', '113',
 '200', '210', '193', '?', '100', '105', '175', '153', '180', '110',
 '72', '86', '70', '76', '65', '69', '60', '80', '54', '208', '155',
 '112', '92', '145', '137', '158', '167', '94', '107', '230', '49',
 '75', '91', '122', '67', '83', '78', '52', '61', '93', '148',
 '129', '96', '71', '98', '115', '53', '81', '79', '120', '152',
 '102', '108', '68', '58', '149', '89', '63', '48', '66', '139',
 '103', '125', '133', '138', '135', '142', '77', '62', '132', '84',
 '64', '74', '116', '82'], dtype=object)

Observation

We check each and every column we got noise in 'horsepower' column

Handling noisy data

```
In [18]: 1 # count the no rows of having noisy data
          2 df[df['horsepower']=='?'].size
          3
```

Out[18]: 54

```
In [19]: 1 df_noise=df[df['horsepower']=='?']
          2 df_noise
```

Out[19]:

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	car name
32	25.0	4	98.0	?	2046	19.0	71	1	ford pinto
126	21.0	6	200.0	?	2875	17.0	74	1	ford maverick
330	40.9	4	85.0	?	1835	17.3	80	2	renault lecar deluxe
336	23.6	4	140.0	?	2905	14.3	80	1	ford mustang cobra
354	34.5	4	100.0	?	2320	15.8	81	2	renault 18i
374	23.0	4	151.0	?	3035	20.5	82	1	amc concord dl

```
In [20]: 1 df_noise.shape
```

Out[20]: (6, 9)

```
In [21]: 1 df_clean=df[df['horsepower']!='?']
          2 df_clean.head()
```

Out[21]:

	mpg	cylinders	displacement	horsepower	weight	acceleration	model year	origin	car name
0	18.0	8	307.0	130	3504	12.0	70	1	chevrolet chevelle malibu
1	15.0	8	350.0	165	3693	11.5	70	1	buick skylark 320
2	18.0	8	318.0	150	3436	11.0	70	1	plymouth satellite
3	16.0	8	304.0	150	3433	12.0	70	1	amc rebel sst
4	17.0	8	302.0	140	3449	10.5	70	1	ford torino

```
In [22]: 1 df_clean.shape
```

```
Out[22]: (392, 9)
```

```
In [23]: 1 df_clean['horsepower'].median()
```

```
Out[23]: 93.5
```

```
In [24]: 1 df['horsepower'].replace("?",df_clean['horsepower'].median(),inplace=True
2 df.shape
```

```
Out[24]: (398, 9)
```

```
In [25]: 1 df['horsepower'].unique()
```

```
Out[25]: array(['130', '165', '150', '140', '198', '220', '215', '225', '190',
'170', '160', '95', '97', '85', '88', '46', '87', '90', '113',
'200', '210', '193', 93.5, '100', '105', '175', '153', '180',
'110', '72', '86', '70', '76', '65', '69', '60', '80', '54', '208',
'155', '112', '92', '145', '137', '158', '167', '94', '107', '230',
'49', '75', '91', '122', '67', '83', '78', '52', '61', '93', '148',
'129', '96', '71', '98', '115', '53', '81', '79', '120', '152',
'102', '108', '68', '58', '149', '89', '63', '48', '66', '139',
'103', '125', '133', '138', '135', '142', '77', '62', '132', '84',
'64', '74', '116', '82'], dtype=object)
```

Observations

We replace noise present in horsepower column with median

Type casting

```
In [26]: 1 df['horsepower']=pd.to_numeric(df['horsepower'])
```

```
In [27]: 1 df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 398 entries, 0 to 397
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   mpg              398 non-null    float64
1   cylinders        398 non-null    int64
2   displacement     398 non-null    float64
3   horsepower       398 non-null    float64
4   weight           398 non-null    int64
5   acceleration     398 non-null    float64
6   model year      398 non-null    int64
7   origin           398 non-null    int64
8   car name        398 non-null    object
dtypes: float64(4), int64(4), object(1)
memory usage: 28.1+ KB
```

Observation

after type casting of horsepower from object to numeric we have now 4 columns are float

3 Missing values

```
In [28]: 1 df.isna().sum()
```

```
Out[28]: mpg          0
cylinders          0
displacement       0
horsepower         0
weight            0
acceleration       0
model year         0
origin            0
car name          0
dtype: int64
```

observations

After checking each column , we found that there is no missing values in the data

4 Duplicates

```
In [29]: 1 df.duplicated().sum()
```

```
Out[29]: 0
```

Observation

After checking the each row , we found that there is no duplicated data

5 Outliers

```
In [30]: 1 df.columns
```

```
Out[30]: Index(['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
               'acceleration', 'model year', 'origin', 'car name'],
              dtype='object')
```

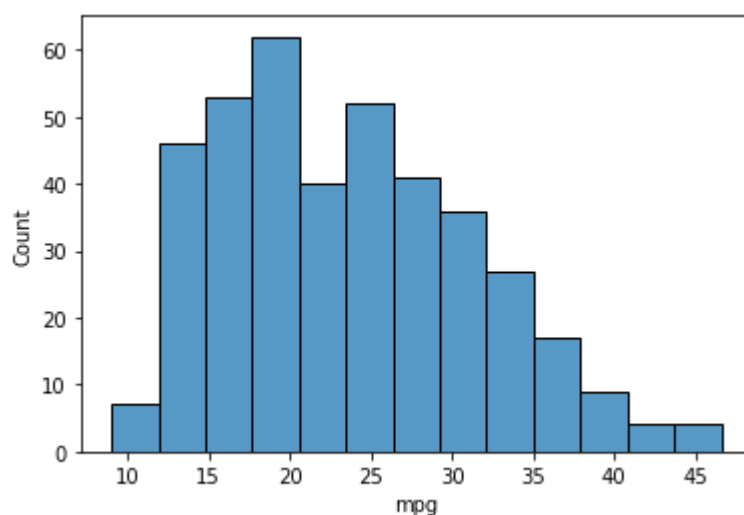
In [31]: 1 df.info()

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 398 entries, 0 to 397
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -
0   mpg              398 non-null    float64
1   cylinders        398 non-null    int64
2   displacement     398 non-null    float64
3   horsepower       398 non-null    float64
4   weight           398 non-null    int64
5   acceleration     398 non-null    float64
6   model year      398 non-null    int64
7   origin           398 non-null    int64
8   car name        398 non-null    object
dtypes: float64(4), int64(4), object(1)
memory usage: 28.1+ KB
```

1. What is the distribution of MPG in the dataset?

In []: 1 # mpg Miles per gallon (MPG), a measure of fuel efficiency for the car. H

In [32]: 1 # To check distribution we use histogram
2 sns.histplot(data=df,x='mpg')
3 plt.show()



Observations

After visualizing the distribution of mpg we get to know that count of cars having mpg between 10-30 are high

2. How many cars are from each origin (USA, Europe, Japan)?

```
In [33]: 1 df['origin'].unique()
```

```
Out[33]: array([1, 3, 2], dtype=int64)
```

```
In [34]: 1 usa=df['origin']==1  
2 usa.sum()
```

```
Out[34]: 249
```

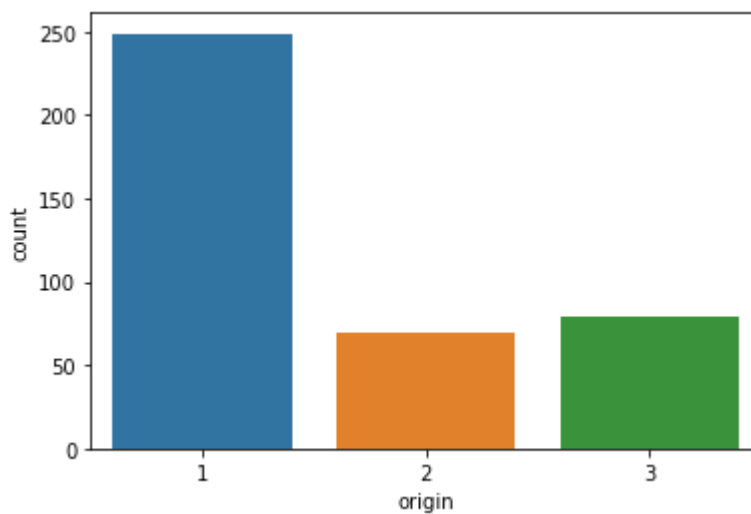
```
In [35]: 1 Europe=df['origin']==2  
2 Europe.sum()
```

```
Out[35]: 70
```

```
In [36]: 1 Japan=df['origin']==3  
2 Japan.sum()
```

```
Out[36]: 79
```

```
In [37]: 1 sns.countplot(data=df,x="origin")  
2 plt.show()
```



Observation

After checking the count of car according to origins, we can tell that USA has more car as compair to Europe and Japan

3. What is proportion of cars are from each origin (USA, Europe, Japan)?

```
In [38]: 1 usa=df['origin']==1  
2 Per_Usa=usa/(usa.shape[0])*100  
3 Per_Usa.sum()
```

Out[38]: 62.56281407035175

```
In [39]: 1 Europe=df['origin']==2  
2 Per_Europe=Europe/(Europe.shape[0])*100  
3 Per_Europe.sum()
```

Out[39]: 17.587939698492463

```
In [40]: 1 Japan=df['origin']==3  
2 Per_Japan=Japan/(Japan.shape[0])*100  
3 Per_Japan.sum()
```

Out[40]: 19.849246231155778

Visulization

```
In [41]: 1 df['origin'].unique()
```

Out[41]: array([1, 3, 2], dtype=int64)

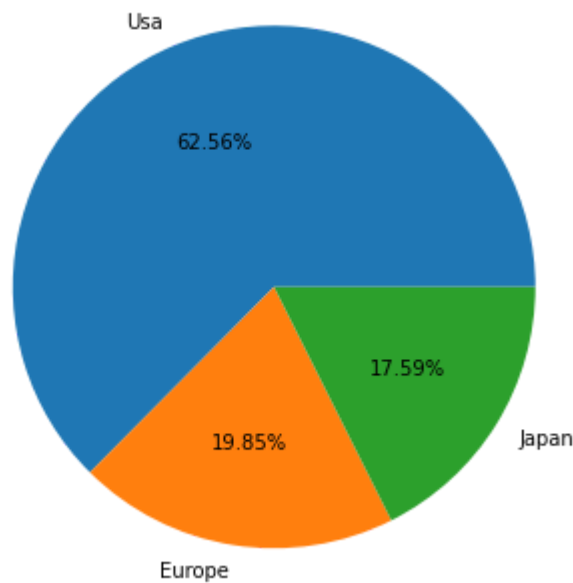
```
In [42]: 1 df['origin'].value_counts().values
```

Out[42]: array([249, 79, 70], dtype=int64)

```
In [43]: 1 df['origin'].value_counts().index
```

Out[43]: Int64Index([1, 3, 2], dtype='int64')

```
In [44]: 1 #plt.pie(df['origin'].value_counts().values, labels=df['origin'].value_cou
2 plt.figure(figsize=(8,6))
3 plt.pie(df['origin'].value_counts().values, labels=['Usa', 'Europe', 'Japan']
4 plt.show()
```



```
In [45]: 1 df.columns
```

```
Out[45]: Index(['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
               'acceleration', 'model year', 'origin', 'car name'],
              dtype='object')
```

3. What percentage of cars belong to each cylinder category?

```
In [46]: 1 df['cylinders'].unique()
```

```
Out[46]: array([8, 4, 6, 3, 5], dtype=int64)
```

```
In [47]: 1 df['cylinders'].value_counts()
```

```
Out[47]: 4    204
         8    103
         6     84
         3     4
         5     3
         Name: cylinders, dtype: int64
```

```
In [48]: 1 Three_Cy=df['cylinders']==3
2 Three_Cy.sum()
```

```
Out[48]: 4
```

```
In [49]: 1 Per_Three_Cy=(Three_Cy.sum()/df['cylinders'].size)*100
        2 Per_Three_Cy
```

Out[49]: 1.0050251256281406

```
In [50]: 1 Four_Cy=df['cylinders']==4
        2 Four_Cy.sum()
```

Out[50]: 204

```
In [51]: 1 Per_Four_Cy=(Four_Cy.sum()/df['cylinders'].size)*100
        2 Per_Four_Cy
```

Out[51]: 51.256281407035175

```
In [52]: 1 #Per_Four_Cy=(Four_Cy.sum()/df['cylinders'].shape[0])*100
        2
```

```
In [53]: 1 Five_Cy=df['cylinders']==5
        2 Per_Five_Cy=(Five_Cy.sum()/df['cylinders'].size)*100
        3 Per_Five_Cy
        4
```

Out[53]: 0.7537688442211055

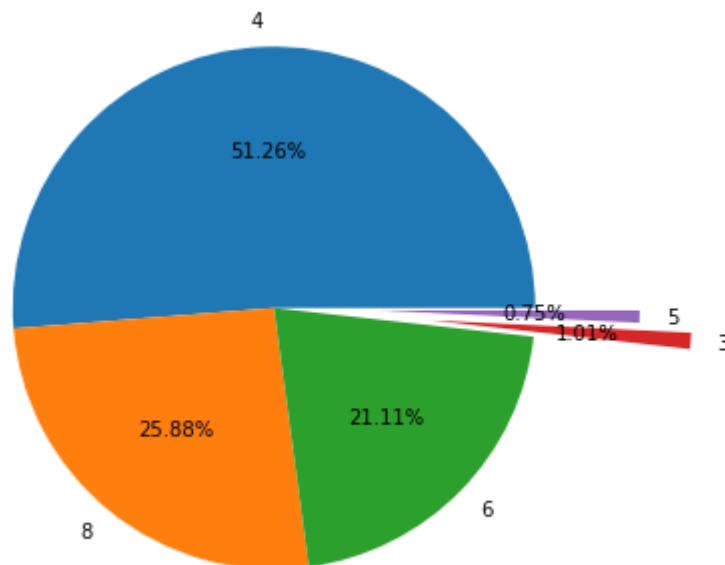
```
In [54]: 1 Six_Cy=df['cylinders']==6
        2 Per_Six_Cy=(Six_Cy.sum()/df['cylinders'].size)*100
        3 Per_Six_Cy
```

Out[54]: 21.105527638190953

```
In [55]: 1 Eight_Cy=df['cylinders']==8
        2 Per_Eight_Cy=(Eight_Cy.sum()/df['cylinders'].size)*100
        3 Per_Eight_Cy
```

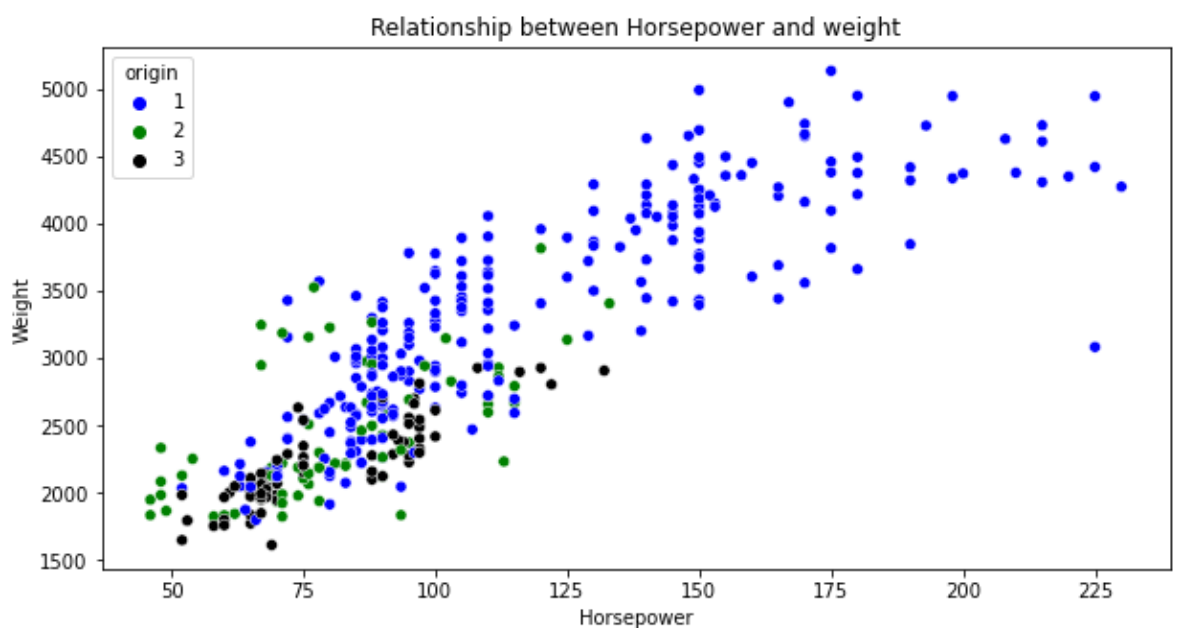
Out[55]: 25.879396984924625

```
In [56]: 1 plt.figure(figsize=(8,6))
2 plt.pie(df['cylinders'].value_counts().values,labels=df['cylinders'].valu
3 #explode
4 plt.show()
```



4. What is the relationship between horsepower and weight of cars?

```
In [57]: 1 plt.figure(figsize=(10,5))
2 sns.scatterplot(data=df,x='horsepower',y='weight',hue='origin',palette=
3 plt.title('Relationship between Horsepower and weight')
4 plt.xlabel('Horsepower')
5 plt.ylabel('Weight')
6 plt.show()
```



Observations

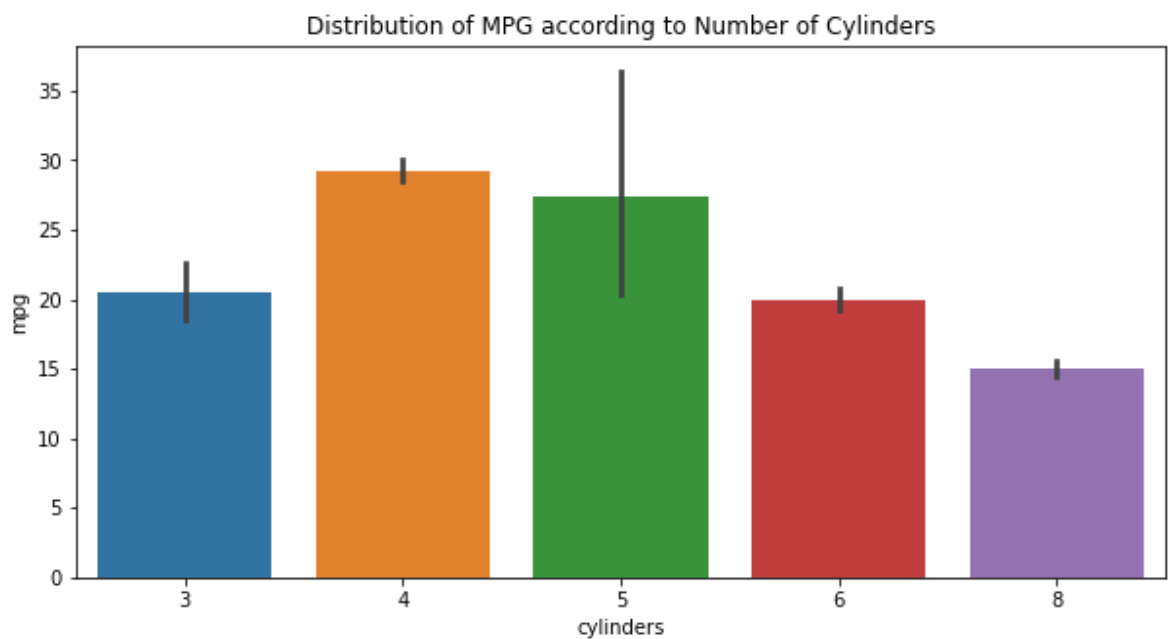
As we see the weight increases

5. How does MPG vary across different numbers of cylinders?

```
In [58]: 1 df.columns
```

```
Out[58]: Index(['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',  
              'acceleration', 'model year', 'origin', 'car name'],  
             dtype='object')
```

```
In [59]: 1 plt.figure(figsize=(10,5))  
2 sns.barplot(data=df,x='cylinders',y='mpg')  
3 plt.title("Distribution of MPG according to Number of Cylinders")  
4 plt.show()
```

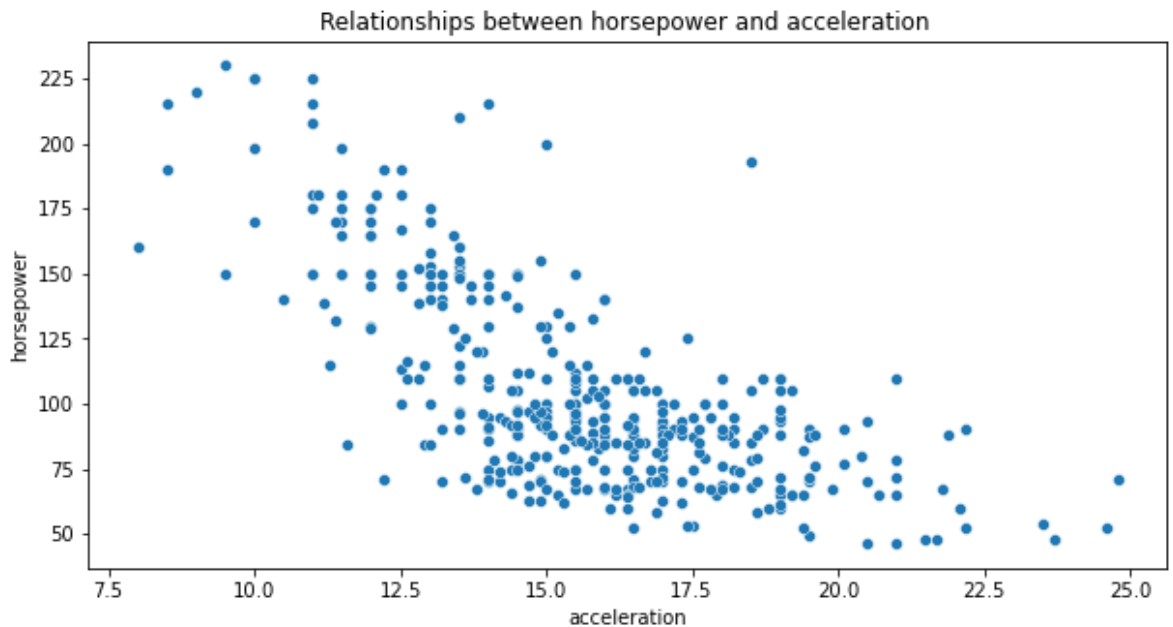


Observations

From analysis we can say that the average milage of car is high for 4 and 5 number cylinders

6. What are the relationships among horsepower, and acceleration ?

```
In [60]: 1 plt.figure(figsize=(10,5))
2 sns.scatterplot(data=df,x='acceleration',y='horsepower')
3 plt.title("Relationships between horsepower and acceleration ")
4 plt.show()
```



```
In [61]: 1 cor=df['acceleration'].corr(df['horsepower'])
2 cor
```

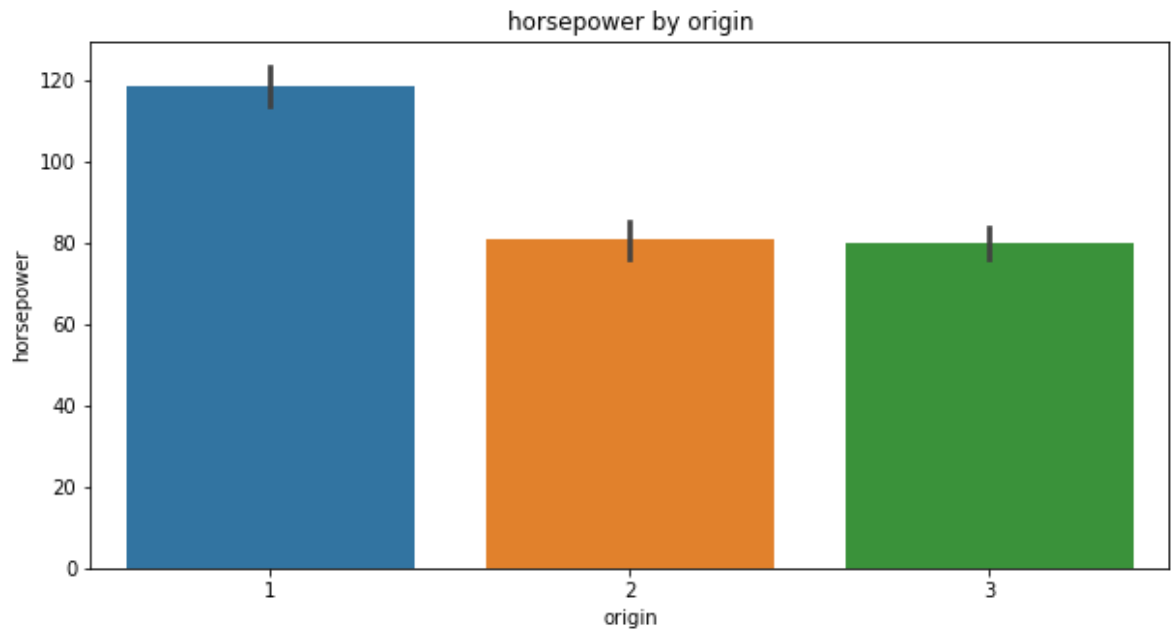
Out[61]: -0.6865897373047709

Observations

As we can see that the horsepower increases the acceleration decreases, we can say that horsepower and acceleration have -ve relationship with each others

7. How does horsepower vary by origin?

```
In [62]: 1 plt.figure(figsize=(10,5))
2         sns.barplot(data=df,x='origin',y='horsepower')
3         plt.title('horsepower by origin')
4         plt.show()
```

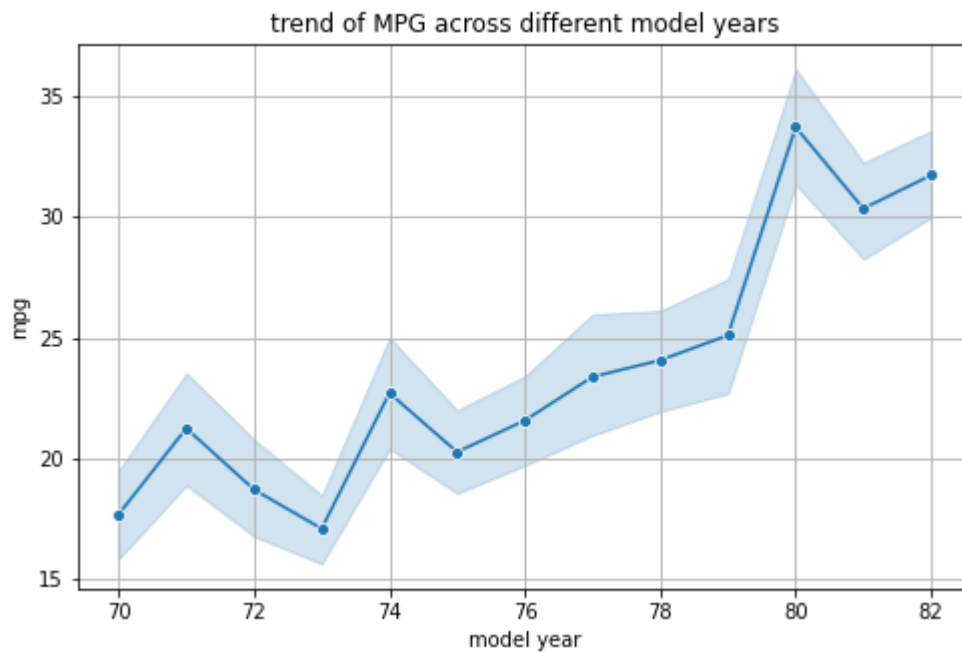


8. What is the trend of MPG across different model years?

```
In [63]: 1 df.columns
```

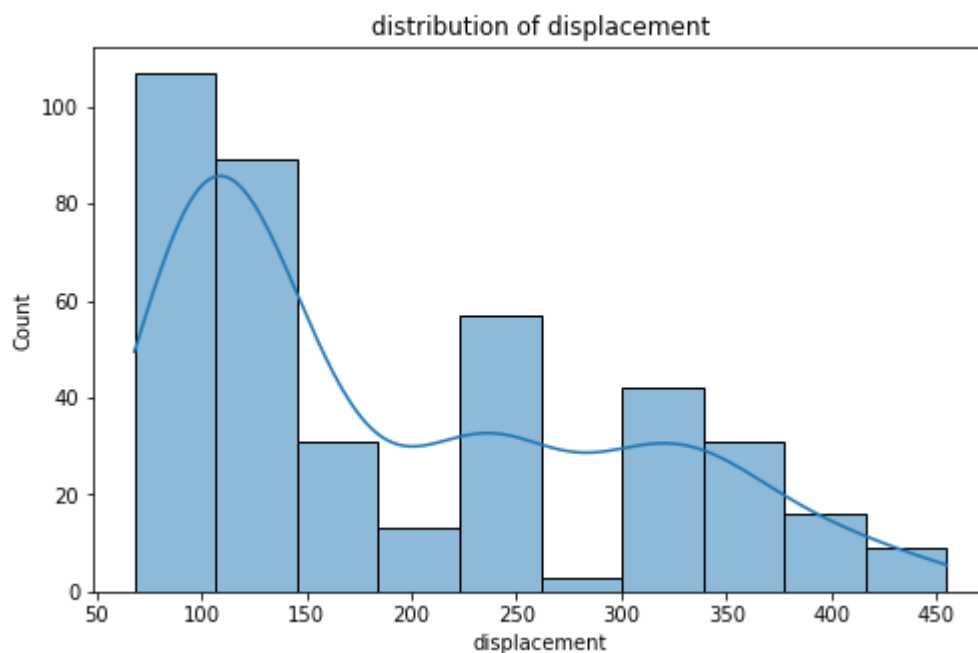
```
Out[63]: Index(['mpg', 'cylinders', 'displacement', 'horsepower', 'weight',
               'acceleration', 'model year', 'origin', 'car name'],
              dtype='object')
```

```
In [64]: 1 plt.figure(figsize=(8,5))
2 sns.lineplot(data=df,x='model year',y='mpg',marker='o',palette='red')
3 plt.title('trend of MPG across different model years')
4 plt.grid()
```



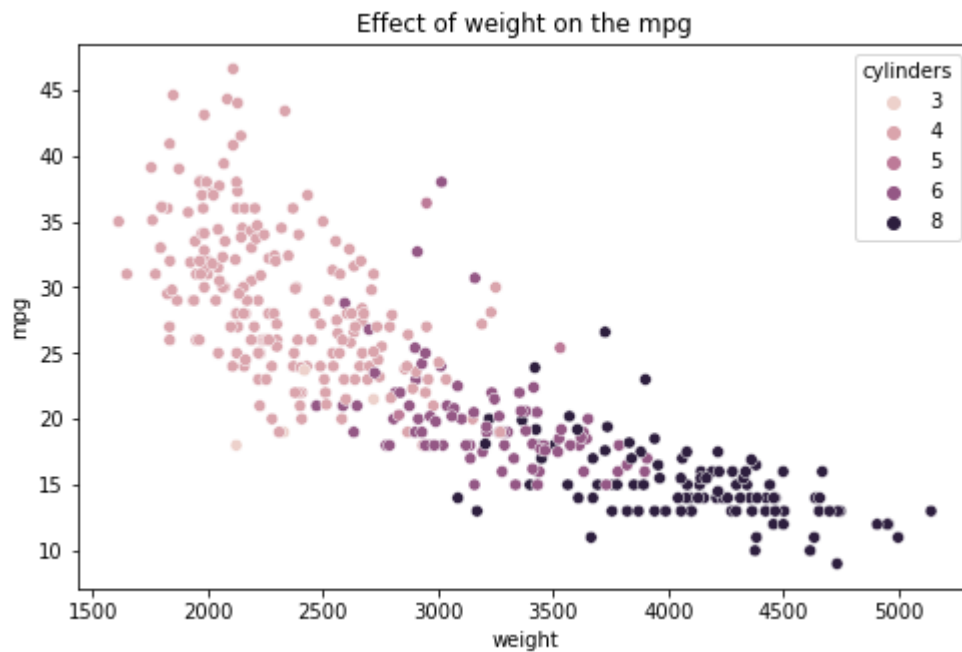
9. What is the distribution of displacement in the dataset?

```
In [74]: 1 plt.figure(figsize=(8,5))
2 sns.histplot(data=df,x='displacement',kde=True)
3 plt.title('distribution of displacement')
4 plt.xlabel("displacement")
5 plt.show()
```



10. How does the weight affect the mpg

```
In [78]: 1 plt.figure(figsize=(8,5))
2 sns.scatterplot(data=df,x='weight',y='mpg',hue='cylinders')
3 plt.title('Effect of weight on the mpg')
4 plt.xlabel("weight")
5 plt.ylabel('mpg')
6 plt.show()
```

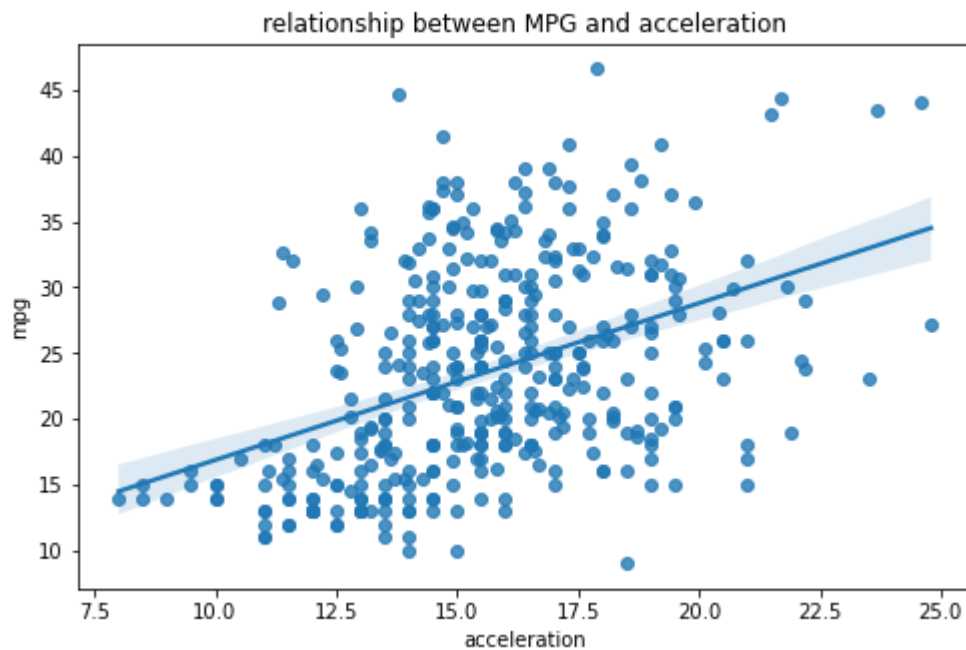


Observation

From analysis we can say that as weight increases the mpg goes down, so we conclude that weight and mpg having -ve relationship

11. What is the relationship between MPG and acceleration?

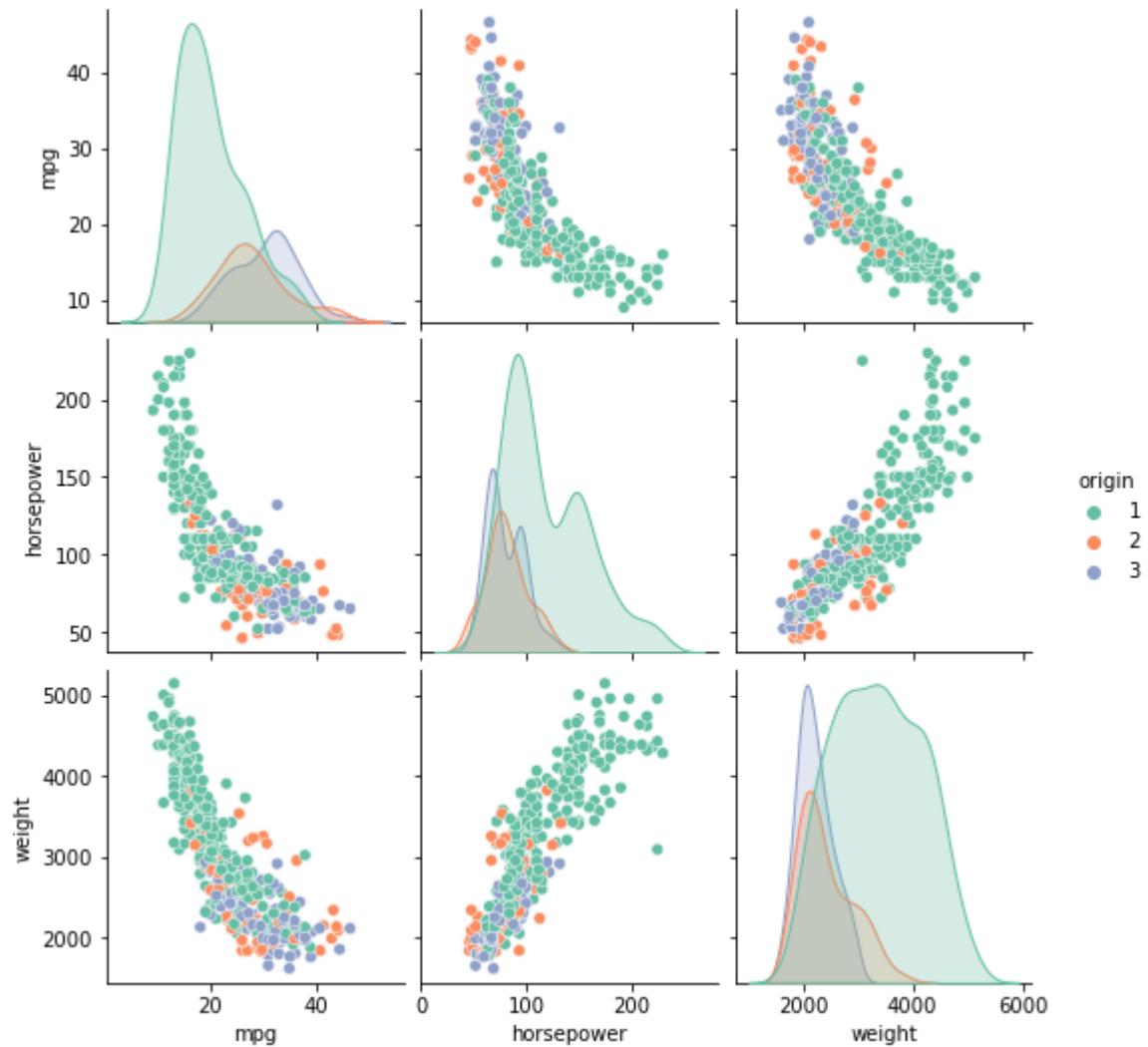
```
In [83]: 1 plt.figure(figsize=(8,5))
2         sns.regplot(data=df,x='acceleration',y='mpg')
3         plt.title('relationship between MPG and acceleration')
4         plt.xlabel("acceleration")
5         plt.ylabel('mpg')
6         plt.show()
```



```
In [ ]: 1
```

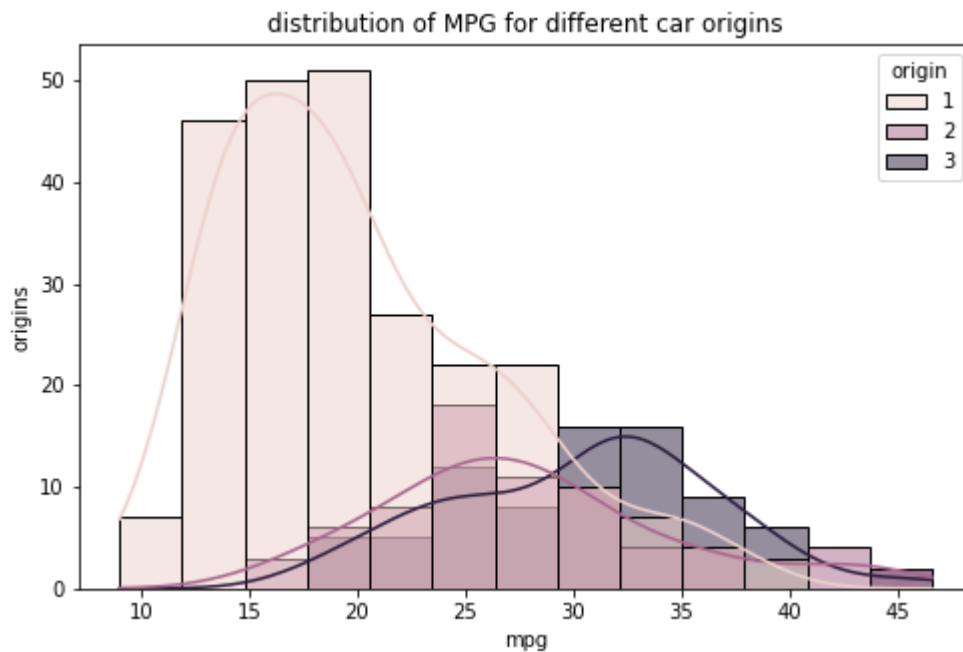
12. What relationships can be inferred between MPG, horsepower, and weight?

```
In [96]: 1 sns.pairplot(data=df, vars=['mpg', 'horsepower', 'weight'], hue='origin', palette='magma')
2 plt.show()
```



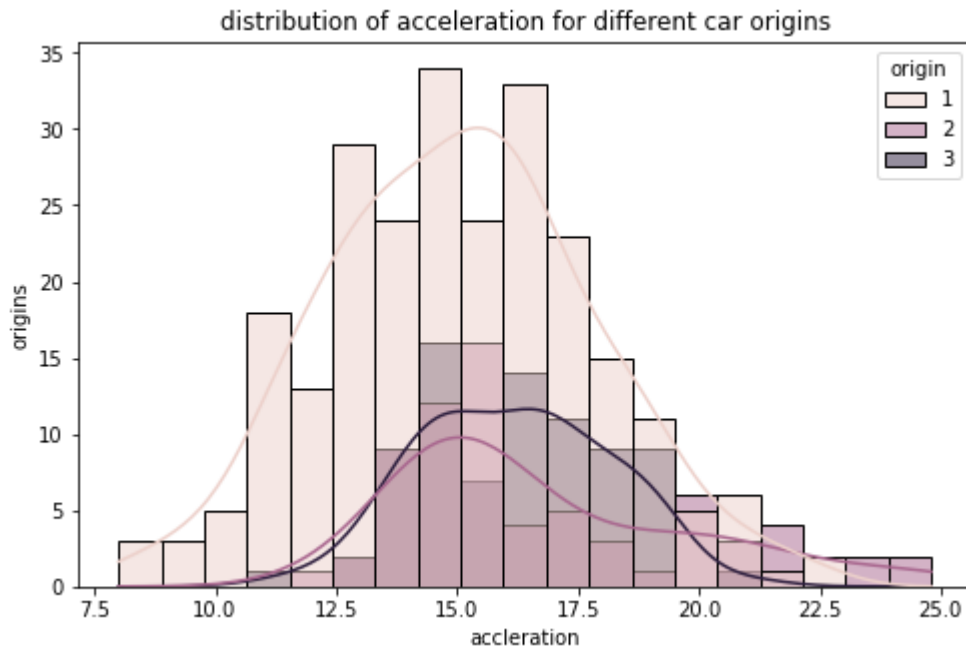
13. What is the distribution of MPG for different car origins?

```
In [101]: 1 plt.figure(figsize=(8,5))
2          sns.histplot(data=df,x='mpg',hue='origin',kde=True)
3          plt.title('distribution of MPG for different car origins')
4          plt.xlabel('mpg')
5          plt.ylabel("origins")
6          plt.show()
```

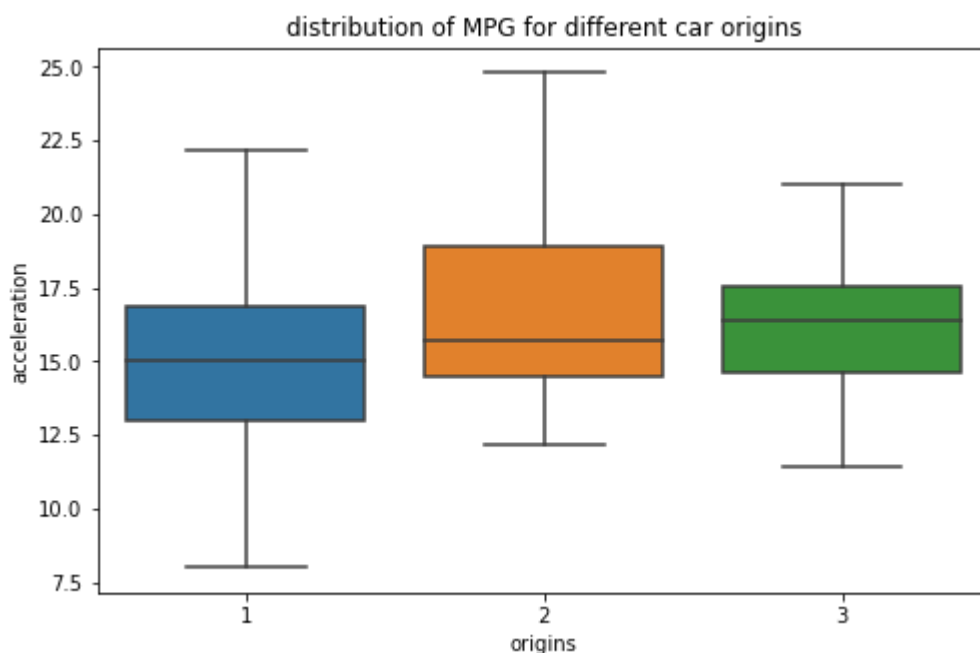


14. What is the distribution of acceleration among various origin of cars?

```
In [106]: 1 plt.figure(figsize=(8,5))
2 sns.histplot(data=df,x='acceleration',hue='origin',kde=True)
3 plt.title('distribution of acceleration for different car origins')
4 plt.xlabel('acceleration')
5 plt.ylabel("origins")
6 plt.show()
```

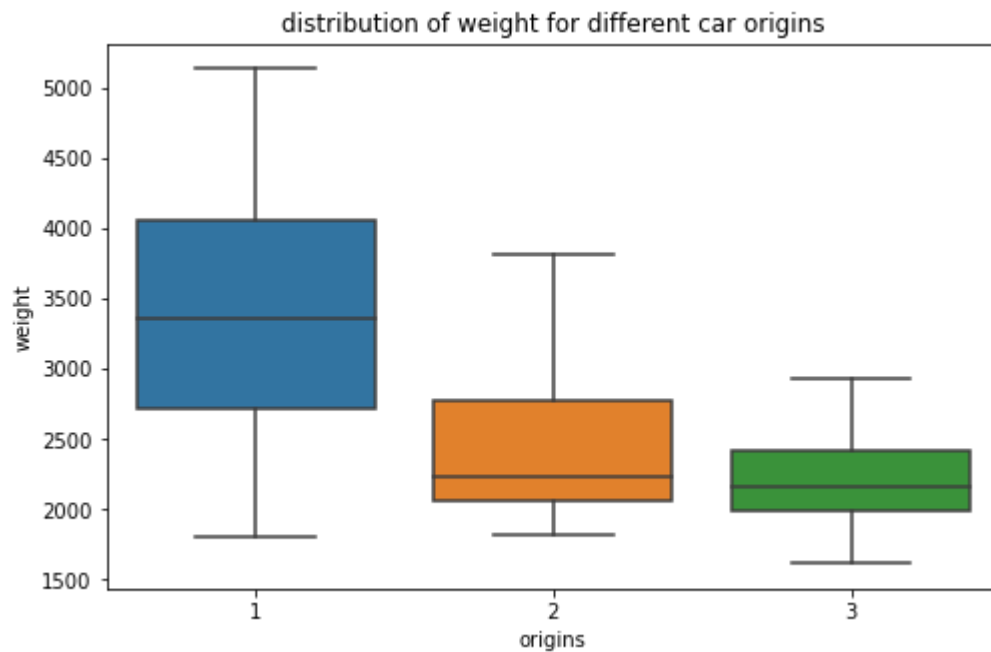


```
In [112]: 1 plt.figure(figsize=(8,5))
2 sns.boxplot(data=df,y='acceleration',x='origin')
3 plt.title('distribution of MPG for different car origins')
4 plt.xlabel('origins')
5 plt.ylabel("acceleration")
6 plt.show()
```



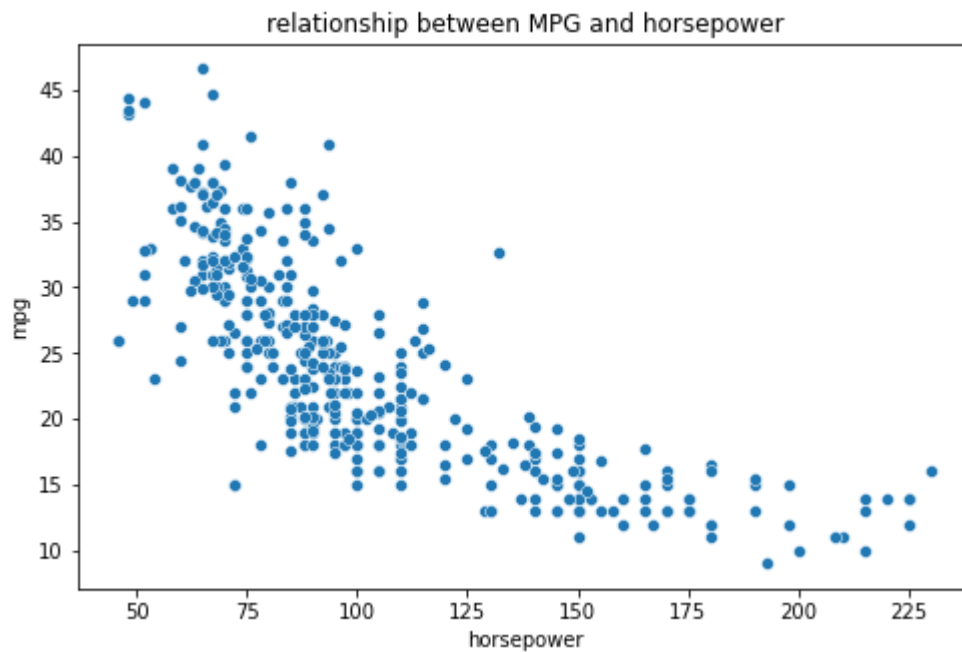
15. How does the weight of cars vary by their origin?

```
In [113]: 1 plt.figure(figsize=(8,5))
2 sns.boxplot(data=df,x='origin',y='weight')
3 plt.title('distribution of weight for different car origins')
4 plt.xlabel('origins')
5 plt.ylabel("weight")
6 plt.show()
```



16. What is the relationship between MPG and horsepower?

```
In [116]: 1 plt.figure(figsize=(8,5))
2 sns.scatterplot(data=df,y='mpg',x='horsepower')
3 plt.title('relationship between MPG and horsepower')
4 plt.xlabel('horsepower')
5 plt.ylabel("mpg")
6 plt.show()
```



```
In [ ]: 1
```